

Chemical Engineering: Don't Runaway

HackRush'25 Submission Report

A try to the solution by:
Aditya Mehta (22110150)
Md. Sibtain Raza (22110148)

Task 1: Conditions, Parameters and kinetic equations required for performing the modelling:

The rate law is second order for n-butyl acrylate with $k(300K) = 0.358 \text{ L/mol.s}$,
 $E_A = 28991.6 \text{ J/mol}$ and Universal Gas Constant $R = 8.314 \text{ J/mol.K}$

Therefore,

$$k = k(300K) * \frac{\exp[-E_A/RT]}{\exp[-E_A/R(300)]} = 4.01 \times 10^3 \times e^{-E_A/RT}$$

$$\Delta H_{Rx}^0 = -64512 \text{ kJ/kmol}$$

$$UA = 5000 \text{ J/s.K}$$

$$m_v = 30 \text{ kg/h} = 1/20 \text{ kg/s}$$

$$-r_A = k C_A^2$$

The reaction is irreversible.

Task 2: Standard and Modified Recipe cases:

Table1: The initial conditions for the standard and modified recipes are:

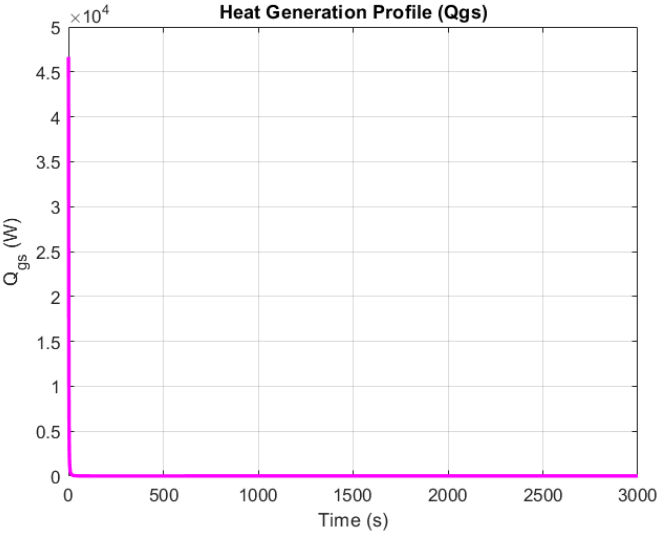
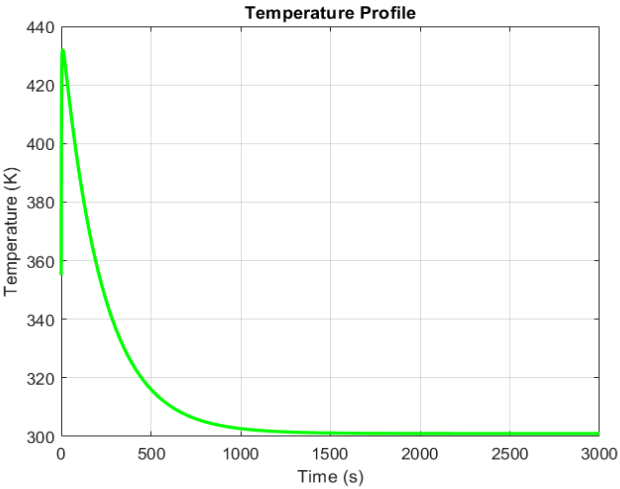
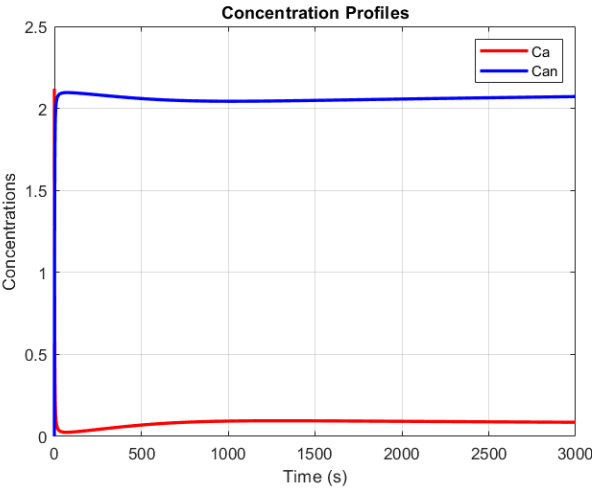
Components	Standard Recipe		Modified Recipe	
	Initial Charge [kmol]	Feed Concentration [kmol / m ³]	Initial Charge [kmol]	Feed Concentration [kmol / m ³]
n-butyl acrylate	1.55	2.22	5.67	3.99
Cyclohexane	2.64	3.52	7.76	1.94
Toluene	2.2	2.92	7.09	2.09
Feed Rate	Vo = 0.73 m ³ ; vo = 0.00015 m ³ / s		Vo = 2.38 m ³ ; vo = 6*10 ⁻⁵ m ³ / s	

Physical Properties				
Components	Molecular Weight [kg / kmol]	Density [kg / m ³]	Sp. Heat [kJ / kmol-K]	Heat of Vaporization [kJ / kg]
n-butyl acrylate	128	870	245.75	278
Cyclohexane	84	750	161.3	360
Toluene	92	840	165.6	360
Poly- butyl acrylate			231	292

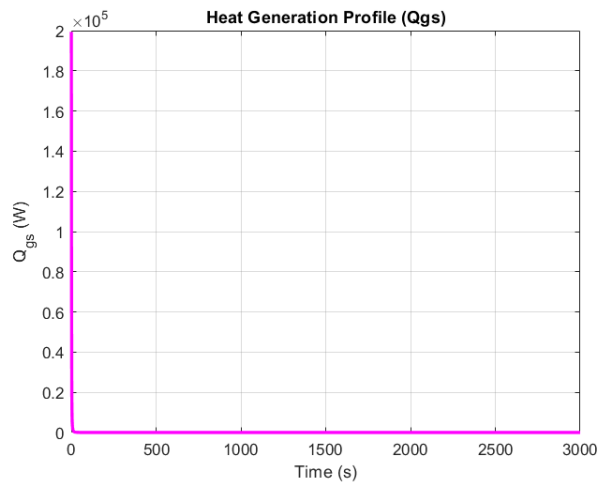
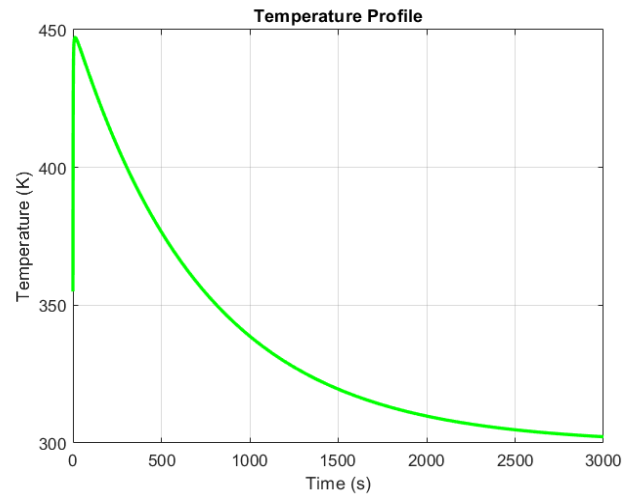
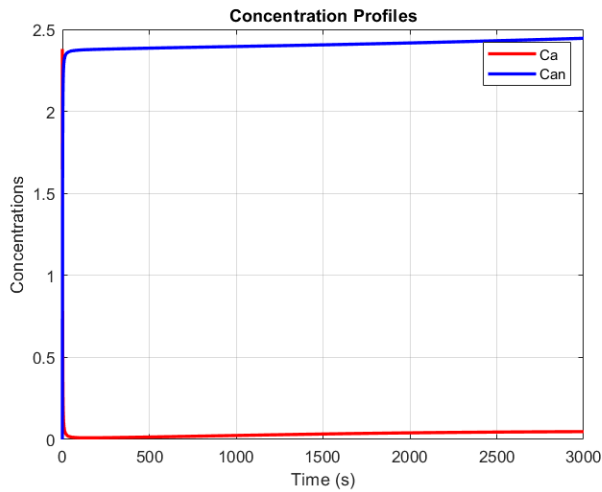
Table2: Physical Properties

Plots:

Standard recipe:



Modified recipe:



Task 3: Exploring different feed strategies and initial charge for the modified recipe

Trial 1:

We initially thought that the runaway is due to more feed and hence we thought of putting an ON-OFF controller in the reactor which measures the temperature (real-time)(as input for the controller) and likewise controls the flow rate for the feed. We decided to set a threshold temperature (say 370 K) (reaction temperature being 355 K) saying that the reactor temperature should not exceed that value and when the temperature starts exceeding that value, the feed inlet should stop. But doing so, we found out that the temperature is exceeding that value, no matter if the feed is on or off. So, this defines that the issue of the runaway is due to the initial charge present in the reactor and less dependent on the feed entering the reactor.

It is possible that the runaway is occurring at some temperature in between 405K (highest temperature in standard scenario) and 445K (highest temperature in modified scenario) since at that temperature, the pressure of the reactor (which we don't know) is not possible. Runaway is occurring due to the impossible pressure rather than taking temperature. We have not dealt with the runaway problems where the runaway is occurring due to pressure. So, let's assume that the runaway is occurring at 405K temperature.

So, the constraint we need to put is that the temperature should not increase 405K in order to stop the runaway. Now, since the runaway is due to the high initial charge present in the reactor. Hence, changing or putting any constraints in the feed of the reactor is not going to help us out in stopping the runaway. Further, if we minimize the initial charge present in the reactor, it would increase the batch time to complete the reaction. (We would have taken the initial charge as 0, it would take very high time for completion of the reaction)

Trail 2: Putting the constraint on the temperature 405 K (let's not let the temperature of the reactor exceed 405K). Further, we shall define a function which gives us the batch time. Further, we will take the help of the **fmincon tool in MATLAB**, we shall proceed. (Normal way would be the hit and trial method in choosing the values of initial charge).

We changing the initial charge as well as not letting the reactor maximum temperature above 405K

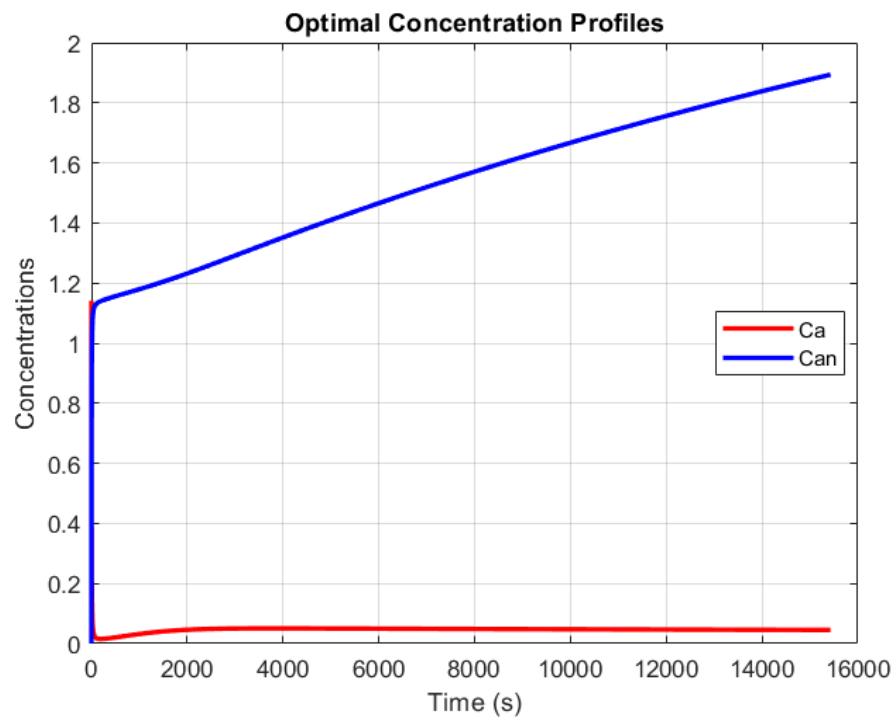
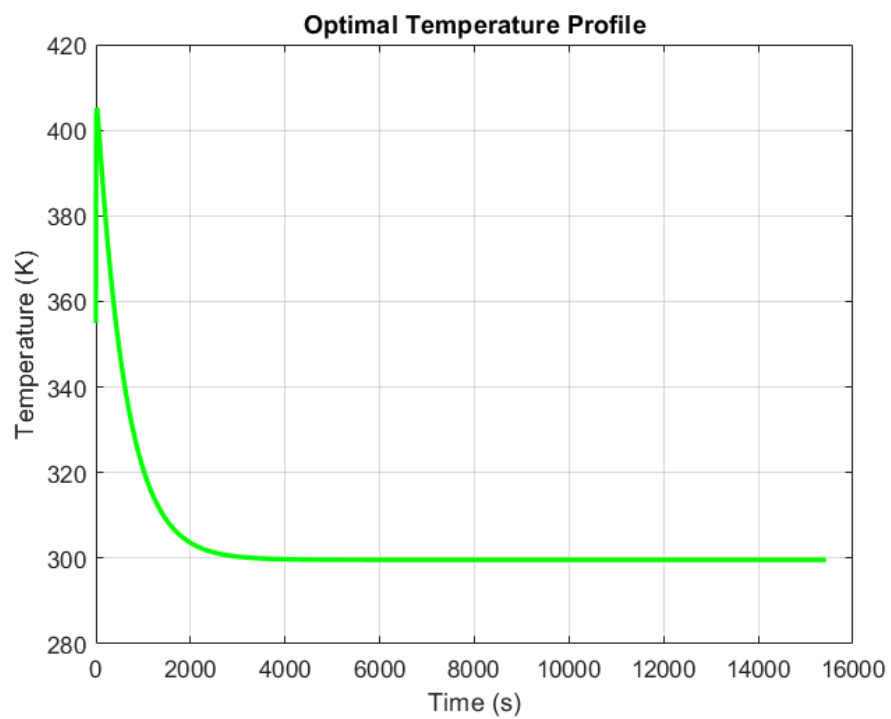
We set the upper bound for the initial charge as a modified recipe value and then optimised the reaction and hence we find out the optimised values are:

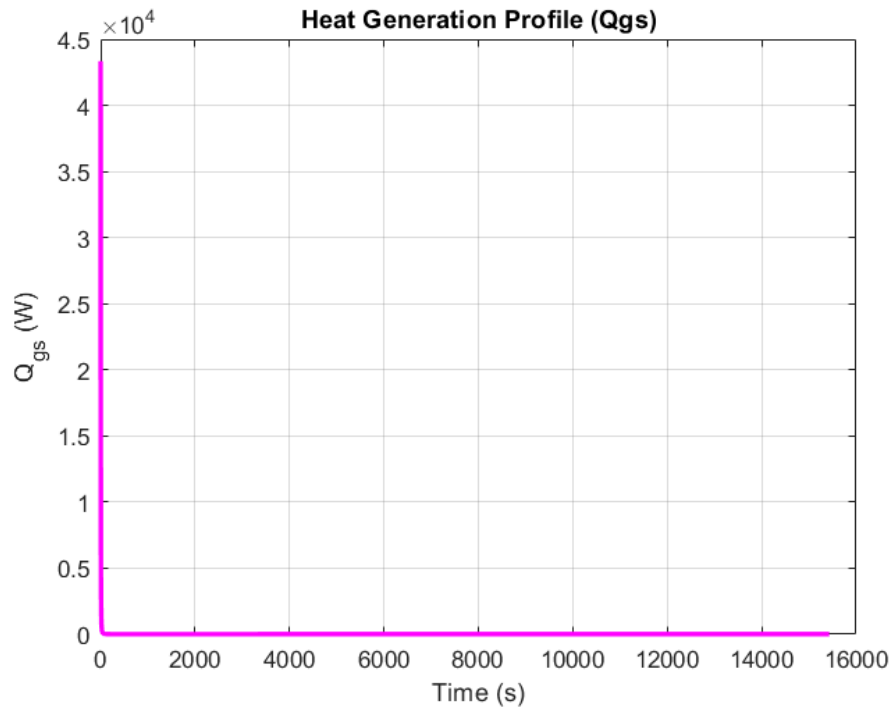
```
% Lower and upper bounds for initial charges (adjust as needed)
lb = [1, 2.64, 2.2];
ub = [2.38, 7.76, 7.09];
```

```
Optimal initial conditions found:
    Ca0 = 1.1413
    Nc0 = 7.7600
    Nt0 = 7.0900
Minimum batch time: 15429.91 sec
```

You can see the plots below that the maximum temperature of the reactor is not exceed 405K.

Plots:





Annexure 1: MATLAB Code for the standard conditions

```
clc; clear; close all;
```

```
% Initial Conditions
```

```
Cao = 2.22;
```

```
Cco = 3.52;
```

```
Cto = 2.92;
```

```
Nc0 = 2.64;
```

```
Nt0 = 2.2;
```

```
Ca0 = 2.12;
```

```
Vo = 0.73;
```

```
vo = 0.00015;
```

```
T0 = 355;
```

```
% Time span
```

```
tspan = [0 3000];
```

```
% Initial values [Ca0, Nc0, Nt0, Can0, T0]
```

```
y0 = [Ca0, Nc0, Nt0, 0, T0];
```

```
% Solve ODEs
```

```
[t, y] = ode45(@(t, y) reactor_odes(t, y, Vo, vo, Cao, Cco, Cto), tspan, y0);
```

```
% Results
```

```
Ca = y(:, 1);
```

```
Can = y(:, 4);
```

```
T = y(:, 5);
```

```
% Plot results
```

```
figure;
```

```
plot(t, Ca, 'r', 'LineWidth', 2);
```

```
hold on;
```

```
plot(t, Can, 'b', 'LineWidth', 2);
```

```
xlabel('Time (s)');
```

```
ylabel('Concentrations');
```

```
legend('Ca', 'Can');
```

```
title('Concentration Profiles');
```

```
grid on;
```

```
figure;
```

```
plot(t, T, 'g', 'LineWidth', 2);
```

```
xlabel('Time (s)');
```

```
ylabel('Temperature (K)');
```

```
title('Temperature Profile');
```

```
grid on;
```

```
% Compute Qgs over time
```

```
Qgs = zeros(length(t), 1);
```

```
for i = 1:length(t)
```

```
    Ca_i = Ca(i);          % Ca at time i
```

```
    T_i = T(i);            % T at time i
```

```
    V_i = Vo + vo * t(i);  % V at time i
```

```
    k_i = 4.01e3 * exp(-29000 / (8.314 * T_i));
```

```
    ra_i = -k_i * Ca_i^2;
```

```
    Qgs(i) = -ra_i * V_i * 64512;
```

```
end
```

```
% Plot Qgs
```

```

figure;
plot(t, Qgs, 'm', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Q_{gs} (W)');
title('Heat Generation Profile (Qgs)');
grid on;

```

```

function dydt = reactor_odes(t, y, Vo, vo, Cao, Cco, Cto)
    % Variables
    Ca = y(1);
    Nc = y(2);
    Nt = y(3);
    Can = y(4);
    T = y(5);

    % Explicit Equations
    V = Vo + (vo*t);
    Na = Ca * V;
    Nan = Can * V;
    Fao = Cao * vo;
    Fco = Cco * vo;
    Fto = Cto * vo;

    k = 4.01e3 * exp(-29000 / (8.314 * T));
    ra = -k * (Ca^2);

    Cpa = 245.75;
    Cpc = 161.30;
    Cpt = 165.60;
    Cpan = 231;

    Qgs = -ra * V * 64512;
    Qrs1 = (Fao*Cpa + Fco*Cpc + Fto*Cpt) * (T - 310);
    Qrs2 = 5 * (T - 298);
    Qrs = Qrs1 + Qrs2 + 8.316;

    NCp = (Na * Cpa) + (Nc * Cpc) + (Nt * Cpt) + (Nan * Cpan);

    % Differential Equations

```

```

dydt(1) = (vo/V) * (Cao - Ca) + ra; % d(Ca)/dt
dydt(2) = Cco * vo;                % d(Nc)/dt
dydt(3) = Cto * vo;                % d(Nt)/dt
dydt(4) = -ra - (Can * (vo/V));    % d(Can)/dt
dydt(5) = (Qgs - Qrs) / NCp;      % d(T)/dt

dydt = dydt(:);
end

```

Annexure 2: MATLAB Code for the modified conditions

```

clc; clear; close all;

% Initial Conditions
Cao = 3.99;
Cco = 1.94;
Cto = 2.09;
Nc0 = 7.76;
Nt0 = 7.09;
Ca0 = 2.38;
Vo = 2.38;
vo = 6e-5;
T0 = 355;

% Time span
tspan = [0 3000];

% Initial values [Ca0, Nc0, Nt0, Can0, T0]
y0 = [Ca0, Nc0, Nt0, 0, T0];

% Solve ODEs
[t, y] = ode45(@(t, y) reactor_odes(t, y, Vo, vo, Cao, Cco, Cto), tspan, y0);

% Results
Ca = y(:, 1);
Can = y(:, 4);
T = y(:, 5);

% Plot results

```

```

figure;
plot(t, Ca, 'r', 'LineWidth', 2);
hold on;
plot(t, Can, 'b', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Concentrations');
legend('Ca', 'Can');
title('Concentration Profiles');
grid on;

```

```

figure;
plot(t, T, 'g', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Temperature (K)');
title('Temperature Profile');
grid on;

```

```

function dydt = reactor_odes(t, y, Vo, vo, Cao, Cco, Cto)
    % Variables
    Ca = y(1);
    Nc = y(2);
    Nt = y(3);
    Can = y(4);
    T = y(5);

    % Explicit Equations
    V = Vo + (vo*t);
    Na = Ca * V;
    Nan = Can * V;
    Fao = Cao * vo;
    Fco = Cco * vo;
    Fto = Cto * vo;

    k = 4.01e3 * exp(-29000 / (8.314 * T));
    ra = -k * (Ca^2);

    Cpa = 245.75;
    Cpc = 161.30;
    Cpt = 165.60;
    Cpan = 231;

```

```

Qgs = -ra * V * 64512;
Qrs1 = (Fao*Cpa + Fco*Cpc + Fto*Cpt) * (T - 310);
Qrs2 = 5 * (T - 298);
Qrs = Qrs1 + Qrs2 + 8.316;

NCp = (Na * Cpa) + (Nc * Cpc) + (Nt * Cpt) + (Nan * Cpan);

% Differential Equations
dydt(1) = (vo/V) * (Cao - Ca) + ra; % d(Ca)/dt
dydt(2) = Cco * vo;                % d(Nc)/dt
dydt(3) = Cto * vo;                % d(Nt)/dt
dydt(4) = -ra - (Can * (vo/V));    % d(Can)/dt
dydt(5) = (Qgs - Qrs) / NCp;      % d(T)/dt

dydt = dydt(:);
end

```

Annexure 3: MATLAB Code for Final Optimization

```

clc; clear; close all;

% Fixed Parameters
Cao = 3.99;
Cco = 1.94;
Cto = 2.09;
Vo = 2.38;
vo = 6e-5;
T0 = 355;
tspan = [0 10000000]; % Simulation time span (sec)

% Optimization Variables: x = [Ca0, Nc0, Nt0]
% Initial guess
x0 = [2.38, 7.76, 7.09];

% Lower and upper bounds for initial charges (adjust as needed)
lb = [1, 2.64, 2.2];
ub = [2.38, 7.76, 7.09];

```

```
% Objective function that simulates the reactor, then returns the batch time when Nan >= target.
objective = @(x) batch_time_obj(x, tspan, Vo, vo, Cao, Cco, Cto, T0);
```

```
% Ensure that the maximum temperature in the reactor does not exceed 405 K.
nonlcon = @(x) temp_constraint(x, tspan, Vo, vo, Cao, Cco, Cto, T0);
```

```
%
options = optimoptions('fmincon','Display','iter','Algorithm','sqp');
[x_opt, fval] = fmincon(objective, x0, [], [], [], [], lb, ub, nonlcon, options);
```

```
fprintf('Optimal initial conditions found:\n');
fprintf(' Ca0 = %.4f\n Nc0 = %.4f\n Nt0 = %.4f\n', x_opt(1), x_opt(2), x_opt(3));
fprintf('Minimum batch time: %.2f sec\n', fval);
```

```
% (Optional) Plot the reactor profiles for the optimal conditions
y0_opt = [x_opt(1), x_opt(2), x_opt(3), 0, T0];
tspan = [0 fval];
[t_opt, y_opt] = ode45(@(t,y) reactor_odes(t, y, Vo, vo, Cao, Cco, Cto), tspan, y0_opt);
```

```
Ca_opt = y_opt(:, 1);
Can_opt = y_opt(:, 4);
T_opt = y_opt(:, 5);
```

```
figure;
plot(t_opt, Ca_opt, 'r', 'LineWidth', 2);
hold on;
plot(t_opt, Can_opt, 'b', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Concentrations');
legend('Ca', 'Can', 'Location','Best');
title('Optimal Concentration Profiles');
grid on;
```

```
figure;
plot(t_opt, T_opt, 'g', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Temperature (K)');
title('Optimal Temperature Profile');
grid on;
```

```

Qgs = zeros(length(t_opt), 1);

for i = 1:length(t_opt)
    Ca_i = Ca_opt(i);      % Ca at time i
    T_i = T_opt(i);        % T at time i
    V_i = Vo + vo * t_opt(i); % V at time i

    k_i = 4.01e3 * exp(-29000 / (8.314 * T_i));
    ra_i = -k_i * Ca_i^2;

    Qgs(i) = -ra_i * V_i * 64512;
end

% Plot Qgs
figure;
plot(t_opt, Qgs, 'm', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Q_{gs} (W)');
title('Heat Generation Profile (Qgs)');
grid on;

% --- Objective Function ---
function f = batch_time_obj(x, tspan, Vo, vo, Cao, Cco, Cto, T0)
    y0 = [x(1), x(2), x(3), 0, T0];
    target_Nan = 6.260775;
    options = odeset('Events', @(t,y) event_Can(t,y,target_Nan));
    [t, y, te, ye, ie] = ode45(@(t,y) reactor_odes(t, y, Vo, vo, Cao, Cco, Cto), tspan, y0, options);

    f = t(end);
end

% --- Nonlinear Constraint Function ---
function [c, ceq] = temp_constraint(x, tspan, Vo, vo, Cao, Cco, Cto, T0)
    y0 = [x(1), x(2), x(3), 0, T0];
    [t, y] = ode45(@(t,y) reactor_odes(t, y, Vo, vo, Cao, Cco, Cto), tspan, y0);
    T = y(:,5);
    % Constraint: maximum temperature - 405 <= 0
    c = max(T) - 405;
    ceq = [];

```

end

```
function [value, isterminal, direction] = event_Can(t, y, target)
    % y(4) corresponds to Can.
    % The event is triggered when value = 0, i.e., when  $\text{Nan}(\text{Can} * V) - \text{target} = 0$ .
    value = y(4)*(2.38 + (6e-5)*t) - target;
    isterminal = 1;
    direction = 1;
end
```

% --- Reactor ODE Function ---

```
function dydt = reactor_odes(t, y, Vo, vo, Cao, Cco, Cto)
```

% Variables

Ca = y(1);

Nc = y(2);

Nt = y(3);

Can = y(4);

T = y(5);

% Explicit Equations

$V = V_o + (v_o * t)$;

$N_a = C_a * V$;

$N_{an} = C_{an} * V$;

$F_{ao} = C_{ao} * v_o$;

$F_{co} = C_{co} * v_o$;

$F_{to} = C_{to} * v_o$;

$k = 4.01e3 * \exp(-29000 / (8.314 * T))$;

$ra = -k * (C_a^2)$;

$C_{pa} = 245.75$;

$C_{pc} = 161.30$;

$C_{pt} = 165.60$;

$C_{pan} = 231$;

$Q_{gs} = -ra * V * 64512$;

$Q_{rs1} = (F_{ao} * C_{pa} + F_{co} * C_{pc} + F_{to} * C_{pt}) * (T - 310)$;

$Q_{rs2} = 5 * (T - 298)$;

$Q_{rs} = Q_{rs1} + Q_{rs2} + 8.316$;

$$NCp = (Na * Cpa) + (Nc * Cpc) + (Nt * Cpt) + (Nan * Cpan);$$

% Differential Equations

$$dydt(1) = (vo/V) * (Cao - Ca) + ra; \quad \% d(Ca)/dt$$

$$dydt(2) = Cco * vo; \quad \% d(Nc)/dt$$

$$dydt(3) = Cto * vo; \quad \% d(Nt)/dt$$

$$dydt(4) = -ra - (Can * (vo/V)); \quad \% d(Can)/dt$$

$$dydt(5) = (Qgs - Qrs) / NCp; \quad \% d(T)/dt$$

$$dydt = dydt(:);$$

end